

Módulo 6 – Capítulo 10 – Sección 01

Self-RAG: el modelo decide cuándo y qué recuperar durante la generación

El RAG estándar tiene una asunción implícita que se acepta sin cuestionamiento en la mayoría de las implementaciones: que siempre es necesario recuperar información del corpus antes de generar la respuesta. Esta asunción es razonable para sistemas donde el corpus cubre exactamente el dominio de las queries, pero ineficiente en sistemas de propósito más amplio donde muchas queries son de conocimiento general que el LLM puede responder correctamente desde sus parámetros —sin recuperación— y donde la recuperación forzada añade latencia, costo y ruido en el contexto sin aportar información nueva. De forma simétrica, el RAG estándar también asume que una única ronda de recuperación es suficiente: recuperar K chunks al inicio y generar la respuesta con ese contexto fijo. Para preguntas que requieren razonamiento multi-step donde la segunda recuperación depende de los resultados de la primera, este modelo es estructuralmente insuficiente.

Self-RAG (Self-Reflective Retrieval-Augmented Generation), propuesto por Asai et al. (2023) de la Universidad de Washington, propone un paradigma distinto: entrenar al modelo para que controle activamente el proceso de recuperación durante la generación. El modelo aprende a emitir **tokens de reflexión** especiales —intercalados en el texto generado— que expresan sus decisiones sobre cuándo y cómo recuperar. El token `[Retrieve]` indica que el modelo necesita información adicional del corpus y activa una búsqueda en ese punto de la generación. Los tokens `[IsREL]` (is relevant), `[IsSUP]` (is supported) e `[IsUSE]` (is useful) permiten al modelo evaluar la relevancia del chunk recuperado, verificar si cada afirmación generada está soportada por el contexto, y evaluar si la respuesta final es útil para la query original. Este mecanismo convierte la recuperación de un proceso externo fijo en una capacidad interna del modelo, integrada en el ciclo de generación de texto.

El resultado de este diseño es doble. Por un lado, el modelo recupera información solo cuando la necesita: si la query es "¿en qué año se fundó Python como lenguaje de programación?", el modelo puede responder directamente desde su conocimiento paramétrico sin emitir ningún token `[Retrieve]`. Por otro lado, el modelo puede recuperar múltiples veces en el transcurso

de una respuesta compleja: para una pregunta de análisis multi-step, puede recuperar información sobre el primer aspecto, generarla, identificar que necesita información adicional sobre un segundo aspecto, recuperar nuevamente, y así hasta completar la respuesta. La auto-verificación mediante `[IsSUP]` añade una capa de groundedness que el RAG estándar no tiene de forma nativa: el modelo evalúa cada afirmación que genera contra el contexto recuperado y puede corregirse o señalar incertidumbre cuando detecta que una afirmación no está soportada.

La **limitación central** de Self-RAG es su dependencia del fine-tuning: el modelo base (Llama 2, Mistral u otro modelo open source) debe ser entrenado específicamente con un dataset que incluye las anotaciones de los tokens de reflexión. Esto requiere recursos de entrenamiento significativos y significa que no puede implementarse directamente con LLMs de API propietaria como GPT-4o o Claude 3.5 Sonnet (que no pueden ser fine-tuneados para emitir tokens de reflexión arbitrarios por los proveedores). La alternativa pragmática para equipos que usan LLMs via API es **Corrective RAG** (ver sección siguiente), que implementa principios similares de auto-verificación mediante prompting en lugar de fine-tuning.

Componentes técnicos de Self-RAG

- **Tokens de reflexión:** cuatro tokens especiales aprendidos durante el fine-tuning: `[Retrieve]` (necesito información del corpus), `[IsREL]` (el chunk recuperado es/no es relevante para mi query actual), `[IsSUP]` (la afirmación que generé está/no está soportada por el contexto), `[IsUSE]` (la respuesta final es/no es útil para la query del usuario).
- **Recuperación condicional vs. recuperación fija:** en RAG estándar, siempre se recuperan K chunks antes de generar; en Self-RAG, el modelo emite o no `[Retrieve]` según su evaluación interna de si necesita información adicional; queries de conocimiento general se responden sin recuperación; queries factuales específicas activan una o más recuperaciones durante la generación.
- **Fine-tuning para Self-RAG:** el modelo base requiere Supervised Fine-Tuning (SFT) sobre un dataset con anotaciones de reflexión; el dataset de entrenamiento se genera usando un LLM fuerte (GPT-4) como crítico que anota cuándo recuperar y evalúa relevancia y soporte en texto de entrenamiento; el proceso es automático pero costoso en tokens de API del LLM crítico.
- **Ventajas de performance documentadas:** Self-RAG supera al RAG estándar en benchmarks de QA que requieren razonamiento multi-step o donde algunas partes de la respuesta requieren recuperación y otras no; mejora Faithfulness al auto-verificar cada

afirmación generada contra el contexto recuperado; reduce el número de tokens de contexto procesados al no recuperar cuando no es necesario.

- **Limitaciones prácticas:** requiere fine-tuning del modelo base (incompatible con LLMs via API sin modificación); la calidad del sistema depende de la calidad de las anotaciones del dataset de entrenamiento; incrementa la latencia para queries que activan múltiples recuperaciones secuenciales durante la generación.
- **Alternativa sin fine-tuning:** para equipos que usan LLMs via API, implementar el ciclo de auto-verificación mediante prompting explícito: instruir al LLM para que, después de generar cada afirmación, verifique si está soportada por el contexto recuperado y señale explícitamente las afirmaciones que no puede verificar.

Nota del Arquitecto: Self-RAG representa el estado del arte en control adaptativo de recuperación, pero la barrera de fine-tuning es real y significativa. Para la mayoría de los equipos de producción que usan LLMs via API, Corrective RAG provee el 80% de los beneficios de auto-verificación sin el costo y la infraestructura de entrenamiento. El caso de uso que más justifica la inversión en Self-RAG es cuando el equipo ya opera su propio modelo fine-tuneado por razones de latencia, costo o privacidad de datos, y puede añadir el fine-tuning de reflexión sobre el modelo que ya gestiona.

Self-RAG es el patrón más invasivo de los avanzados porque requiere intervenir el propio modelo. El siguiente patrón —Adaptive RAG— produce mejoras de eficiencia significativas sin necesidad de modificar el modelo generador.

Módulo 6 – Capítulo 10 – Sección 02

Adaptive RAG: clasificación de consultas y routing a estrategias de recuperación

El RAG estándar aplica el mismo pipeline de recuperación a todas las queries: el mismo modelo de embedding, el mismo K, el mismo esquema híbrido de búsqueda, el mismo reranker. Esta uniformidad es práctica para implementaciones iniciales, pero ineficiente cuando el sistema atiende una distribución amplia y heterogénea de tipos de consultas. Una pregunta factual simple como "¿cuál es la dirección del headquarters de la empresa?" tiene requerimientos de recuperación radicalmente distintos a una pregunta de análisis comparativo como "¿qué diferencias hay entre el plan básico y el plan enterprise en términos de SLAs de soporte y política de datos?": la primera se responde óptimamente con K=1–2 y búsqueda directa; la segunda requiere K=10–15 con diversificación de fuentes para cubrir múltiples dimensiones de comparación. Aplicar K=10 con reranking y búsqueda híbrida a la pregunta factual es un gasto innecesario de latencia y costo; aplicar K=3 sin MMR a la pregunta comparativa produce una respuesta incompleta. Adaptive RAG resuelve esta ineficiencia enrutando cada query al pipeline de recuperación más apropiado según su naturaleza.

El componente central del Adaptive RAG es el **clasificador de queries**: un modelo ligero que asigna cada query a una categoría de tipo antes de que el pipeline de recuperación se ejecute. Las categorías típicas —calibradas según el dominio del sistema— incluyen: `simple_factual` (respuesta en un único chunk, respuesta directa), `comparative` (comparación de múltiples entidades o conceptos, requiere diversidad de chunks), `multi_hop` (respuesta requiere encadenar información de múltiples fuentes), `temporal` (información que cambia en el tiempo, requiere filtrado por fecha), `conversational` (referencia a turnos anteriores de la conversación, puede no requerir recuperación adicional), y `no_retrieval` (conocimiento general que el LLM puede responder desde sus parámetros). El clasificador puede implementarse de dos formas: como un LLM pequeño con un prompt de clasificación en pocas líneas —GPT-4o-mini o Claude 3 Haiku añaden 200–300ms de latencia pero tienen alta accuracy de clasificación— o como un modelo de clasificación de texto fine-tuneado sobre

BERT o DistilBERT que clasifica en menos de 10ms sin llamada a API. La elección depende del volumen de queries y del presupuesto de latencia del sistema.

Cada categoría de query se mapea a un **pipeline de recuperación específico**: `simple_factual` usa búsqueda vectorial directa con $K=3$ sin reranker; `comparative` usa búsqueda híbrida con $K=10-15$ y MMR para diversificación de fuentes; `multi_hop` usa múltiples búsquedas encadenadas con la primera recuperación orientando la query de la segunda; `temporal` añade un filtro de metadatos de fecha a la búsqueda vectorial para recuperar solo documentos del rango temporal relevante; `conversational` inspecciona el historial de la conversación y decide si la respuesta está ya disponible sin recuperación adicional; `no_retrieval` omite completamente la fase de búsqueda y envía la query directamente al generador. El **router** de pipeline es el componente que recibe la clasificación y dirige la query al pipeline correspondiente; puede implementarse como un diccionario de `{categoria: pipeline_function}` o como un LangGraph con edges condicionales que seleccionan el path de ejecución según la categoría.

La **evaluación del routing** es un paso frecuentemente omitido pero necesario para justificar la complejidad adicional del Adaptive RAG. El equipo debe construir un subset del golden dataset etiquetado por tipo de query y medir si el pipeline adaptado para cada tipo supera al pipeline genérico para ese tipo. Si el pipeline de `comparative` con $K=15$ y MMR produce un Recall@5 de 0.78 para queries comparativas, mientras el pipeline genérico con $K=5$ produce 0.61 para las mismas queries, el routing justifica su overhead. Si la diferencia es de 2–3 puntos, el overhead del clasificador (latencia + costo) puede no justificarse.

Componentes del Adaptive RAG

- **Clasificador de queries:** LLM pequeño (200–300ms, alta accuracy) o modelo de clasificación fine-tuneado ($< 10\text{ms}$, requiere datos de entrenamiento etiquetados); categorías típicas: `simple_factual`, `comparative`, `multi_hop`, `temporal`, `conversational`, `no_retrieval`; el número de categorías debe calibrarse según la distribución real de queries del sistema —añadir una categoría solo si tiene al menos el 5–10% del tráfico.
- **Pipelines por categoría:** `simple_factual` → $K=3$, búsqueda vectorial; `comparative` → $K=10-15$, búsqueda híbrida + MMR; `multi_hop` → recuperación iterativa con encadenamiento de queries; `temporal` → búsqueda vectorial + filtro de metadatos de fecha; `conversational` → inspección del historial + recuperación opcional; `no_retrieval` → generación directa sin búsqueda.

- **Router de pipeline:** diccionario `{categoria: pipeline_function}` o LangGraph con edges condicionales; el router debe ser stateless y de latencia baja (no debe añadir overhead significativo al pipeline); loggear la categoría asignada a cada solicitud para análisis de distribución de queries en producción.
- **Query complexity estimation como alternativa:** en lugar de categorías discretas, estimar una complejidad de 1–3 con heurísticas (longitud de la query, presencia de palabras de comparación como "diferencia", "vs", "ventajas", presencia de términos temporales); escalar agresivamente el pipeline según la complejidad estimada; más simple de implementar que la clasificación categoría pero menos preciso.
- **Fallback strategy para baja confianza:** cuando el clasificador tiene baja confianza ($\text{score} < 0.6$), usar el pipeline más robusto como default conservador (búsqueda híbrida + reranking + $K=10$); registrar los casos de baja confianza para análisis periódico y mejora del clasificador.
- **Evaluación del adaptive routing:** subset del golden dataset etiquetado por tipo de query; medir Recall@K de la estrategia adaptada vs. la estrategia genérica para cada tipo de query; declarar el routing justificado solo cuando la mejora promedio supera el overhead de latencia y costo del clasificador.

Nota del Arquitecto: Adaptive RAG tiene mayor valor en sistemas de propósito amplio —asistentes empresariales que reciben preguntas de soporte técnico, análisis de contratos, consultas de política interna y preguntas de conocimiento general en el mismo sistema. En sistemas de dominio estrecho donde el 90% de las queries son del mismo tipo, la clasificación es inútil y añade latencia sin beneficio. Analizar la distribución de tipos de query en el golden dataset antes de implementar Adaptive RAG; si la distribución es uniforme entre tipos, el sistema se beneficia significativamente del routing adaptativo.

Adaptive RAG mejora la eficiencia del sistema sin modificar la arquitectura del modelo generador. El siguiente patrón —Corrective RAG— mejora la calidad de las respuestas añadiendo una capa de evaluación entre el retriever y el generador.

Módulo 6 – Capítulo 10 – Sección 03

Corrective RAG: evaluación de la relevancia del contexto antes de generar

Hay una asunción silenciosa en el pipeline RAG estándar que produce una de las categorías de error más frecuentes en producción: la asunción de que los chunks recuperados por el retriever son relevantes para responder la query del usuario. En la práctica, el modelo de embedding devuelve los K chunks con mayor similitud coseno a la representación vectorial de la query, pero similitud coseno alta no garantiza relevancia semántica real. Cuando el corpus no contiene información directamente relevante para una query, el retriever devuelve los chunks "menos malos" —los más similares a la query de entre los disponibles— y el LLM generador recibe esos chunks como si fueran contexto válido. El resultado predecible es una alucinación: el LLM genera una respuesta que suena coherente pero que no está fundamentada en el corpus porque el corpus no tenía la información solicitada.

Corrective RAG (CRAG), propuesto por Yan et al. (2024), inserta una capa de evaluación de relevancia entre el retriever y el generador para interceptar exactamente esta categoría de error. El principio es simple pero poderoso: antes de pasar los chunks recuperados al LLM generador, un **evaluador de relevancia** puntúa cada chunk según su relevancia real para la query. Si los chunks recuperados superan el umbral de relevancia, el pipeline procede normalmente. Si todos los chunks caen por debajo del umbral —señal de que el corpus no tiene información relevante— el sistema activa una **estrategia de fallback** en lugar de generar con contexto irrelevante: puede buscar información complementaria en la web (Tavily API, SerpAPI), consultar un corpus alternativo, o responder al usuario que no hay información suficiente en el corpus disponible. La evaluación de relevancia actúa como un guardarraíl que previene que los chunks de baja calidad contaminen la generación.

La **implementación del evaluador de relevancia** admite distintos niveles de sofisticación según las restricciones del sistema. El nivel más simple —y suficiente para la mayoría de los casos de uso en producción— es un score threshold sobre la similitud coseno del top chunk recuperado: si el chunk con mayor similitud coseno tiene un score menor a 0.65 (calibrado sobre el golden dataset), el sistema interpreta que el corpus no tiene información

relevante y responde con un mensaje de no-información. Este nivel no requiere ningún componente adicional: el score de similitud coseno ya está disponible en la respuesta de cualquier base de datos vectorial. El nivel intermedio usa un LLM pequeño con un prompt de clasificación binaria ("¿este chunk contiene información relevante para responder la siguiente pregunta?: sí/no") aplicado a cada chunk recuperado. El nivel completo del paper de CRAG incluye además la búsqueda web de fallback y el proceso de knowledge refinement que combina y filtra los resultados internos y externos antes de construir el contexto final.

La decisión de implementar el nivel simple, intermedio o completo de CRAG debe ser guiada por la tasa de queries sin cobertura en el corpus del sistema —medida como el porcentaje de queries del golden dataset donde ningún chunk recuperado es relevante. Si esa tasa es baja (< 5%), el nivel simple puede ser suficiente. Si es alta (> 15%), el fallback web del nivel completo puede justificarse para mantener la utilidad del sistema incluso para queries fuera del dominio del corpus. El overhead de latencia del evaluador de relevancia a nivel intermedio es de 50–200ms por chunk evaluado, lo que para $K=5$ representa 250–1000ms adicionales antes de la generación; este overhead debe compararse con el beneficio de Faithfulness que produce.

Aspectos técnicos de Corrective RAG

- **Score threshold como evaluador simple:** umbral de similitud coseno del top chunk (0.60–0.75 según el modelo de embedding); si el top chunk cae por debajo del umbral, responder con mensaje de no-información; calibrar el umbral sobre el golden dataset maximizando la F1 entre precisión (no falsos positivos de no-información) y recall (no alucinaciones por contexto irrelevante).
- **Evaluador LLM de relevancia (nivel intermedio):** LLM pequeño (GPT-4o-mini, Claude 3 Haiku) con prompt "¿Este fragmento contiene información útil para responder la siguiente pregunta? Responde solo 'sí' o 'no'."; aplicar a cada chunk recuperado; overhead de 50–200ms por chunk; mayor precisión que el threshold de coseno para relevancia inferencial (cuando la relevancia no es léxica sino conceptual).
- **Decisión según evaluación:** si al menos un chunk tiene score de relevancia alto → proceder con generación usando solo los chunks relevantes; si todos los chunks son irrelevantes → activar fallback (búsqueda web, corpus alternativo, o mensaje de no-información); si solo algunos chunks son relevantes → proceder con los relevantes, descartar los irrelevantes.
- **Búsqueda web de fallback:** Tavily API o SerpAPI para búsqueda en tiempo real cuando el corpus interno no cubre la query; los resultados web se procesan (extracción de

contenido de las URLs top, chunking básico) y se añaden al contexto con una nota al LLM de que la información proviene de fuentes externas; incluir una indicación al usuario de que la respuesta usa fuentes externas.

- **Knowledge refinement:** proceso del paper CRAG completo que combina chunks relevantes del corpus interno con resultados web de fallback, aplica un filtro adicional para eliminar información contradictoria, y construye el contexto final consolidado antes de la generación; implementación más compleja pero produce el contexto de mayor calidad.
- **Implementación lightweight recomendada para producción:** threshold de similitud coseno + mensaje de no-información cuando threshold no se alcanza + evaluador LLM para los casos borderline (scores entre 0.55 y 0.70); esta versión captura el 80% del beneficio del CRAG completo con el 20% de la complejidad de implementación.

***Nota del Arquitecto:** La versión lightweight de CRAG —el threshold de similitud coseno con fallback a no-información— debería ser un componente estándar en cualquier sistema RAG de producción, no un patrón "avanzado". El guardarraíl de relevancia previene la categoría de alucinación más frecuente: las causadas por recuperación fallida de un corpus sin cobertura. Implementarlo requiere menos de 10 líneas de código adicional sobre el pipeline estándar y tiene impacto medible en Faithfulness. El CRAG completo con búsqueda web de fallback es el paso siguiente para sistemas que necesitan responder queries fuera de su corpus base.*

Corrective RAG añade evaluación y corrección antes de la generación. FLARE, el siguiente patrón, introduce evaluación y recuperación adaptativa durante la generación, abordando la incertidumbre del modelo en tiempo real.

Módulo 6 – Capítulo 10 – Sección 04

FLARE: recuperación anticipada basada en predicciones del modelo

El RAG estándar y Corrective RAG comparten una característica: la recuperación ocurre antes de que el modelo comience a generar la respuesta. Esta arquitectura "pre-retrieval" tiene sentido para queries donde el conjunto de información necesaria puede anticiparse desde la query inicial, pero muestra sus limitaciones cuando la generación de la respuesta va descubriendo sub-preguntas que no eran predecibles desde el principio. Una respuesta que comienza describiendo el contexto regulatorio de una industria puede encontrarse, a mitad de la generación, necesitando información específica sobre una empresa o regulación que no fue anticipada en la query original y que los chunks recuperados inicialmente no cubren. En ese punto, el modelo estándar solo puede continuar generando con la información disponible, lo que puede producir afirmaciones no fundamentadas para esa parte específica de la respuesta.

FLARE (Forward-Looking Active Retrieval, Jiang et al. 2023) propone un mecanismo distinto: el modelo genera una **respuesta provisional tentativa** para identificar qué partes de la respuesta son inciertas, y activa recuperación adicional específicamente para esos segmentos de incertidumbre. El mecanismo central es la inspección de las **log-probabilidades de los tokens generados**: cuando el LLM genera texto, cada token tiene una probabilidad asignada que refleja la confianza del modelo en esa elección. Tokens con log-probabilidad baja (por debajo de un umbral calibrado, típicamente -1.0 a -2.0) indican que el modelo está "adivinando" —hay incertidumbre genuina sobre el valor correcto de esa parte de la respuesta. FLARE identifica estos segmentos de baja confianza, formula queries de recuperación a partir del contexto alrededor del segmento incierto, recupera información específica del corpus para clarificar esos segmentos, y regenera la respuesta con la información adicional obtenida.

La intuición detrás de FLARE es que los modelos de lenguaje exhiben una señal calibrativa genuina en sus probabilidades de token: cuando el modelo genera un nombre propio, una fecha específica o un número estadístico con alta confianza, generalmente es porque ese dato está bien representado en su entrenamiento. Cuando lo genera con baja confianza, está

señalando que necesita información externa para afirmar ese valor correctamente. FLARE convierte esa señal interna del modelo en una query de recuperación explícita y dirigida, haciendo que la búsqueda sea mucho más precisa que la búsqueda basada en la query inicial completa: en lugar de buscar chunks relacionados con el tema general de la query, FLARE busca chunks con información específica sobre el sub-aspecto puntual que el modelo identifica como incierto.

Las **limitaciones prácticas de FLARE** son igualmente importantes. El overhead computacional es significativo: la generación tentativa produce tokens de output que se descartan si el segmento se regenera con información adicional, y el ciclo puede ejecutarse múltiples veces para una sola respuesta. El costo total de generación puede ser 2–4 veces mayor que el RAG estándar para queries complejas. Además, FLARE requiere acceso a las log-probabilidades de los tokens generados, que no todos los proveedores de LLMs exponen de forma directa: la API de OpenAI expone `logprobs=True` en sus endpoints de completions; la API de Anthropic no expone log-probabilidades de tokens individuales, lo que requiere adaptaciones como usar modelos locales (Llama, Mistral) vía Ollama o vLLM, o implementar una aproximación donde se instruye al modelo a expresar su incertidumbre explícitamente mediante tokens especiales en el texto.

Componentes técnicos de FLARE

- **Lookahead generation con logprobs:** generar una primera versión tentativa de la respuesta usando el campo `logprobs=True` en la API del LLM; inspeccionar la log-probabilidad de cada token generado; identificar segmentos (secuencias de tokens consecutivos) con log-probabilidad < threshold (-1.0 a -2.0 según calibración sobre el golden dataset del sistema).
- **Query formulation desde el segmento incierto:** construir la query de recuperación a partir del contexto inmediato alrededor del segmento de baja probabilidad; si el segmento incierto es "1952" en la oración "El tratado fue ratificado en [1952]", la query de recuperación puede ser "¿en qué año fue ratificado el tratado X?"; la precisión de la query depende del contexto disponible alrededor del segmento.
- **Recuperación iterativa:** FLARE puede ejecutar múltiples rondas de generación + recuperación para una misma query; en cada ronda, los tokens de baja probabilidad identificados orientan nuevas recuperaciones más específicas; detener el proceso cuando no hay segmentos con log-probabilidad por debajo del threshold, o cuando se alcanza el número máximo de iteraciones (2–3 es el rango práctico).

- **Costo computacional:** overhead de 2–4x respecto al RAG estándar para queries que activan múltiples iteraciones; los tokens de la generación tentativa descartada se facturan como tokens de output; FLARE es económicamente apropiado para dominios donde el costo de una alucinación (medicina, legal, regulatorio) supera ampliamente el costo de la recuperación adicional.
- **FLARE vs. CRAG (complementariedad):** FLARE detecta incertidumbre durante la generación y recupera información específica para resolverla; CRAG detecta irrelevancia del contexto recuperado antes de la generación; son complementarios en sistemas de alta precisión: CRAG filtra la entrada del generador, FLARE corrige la generación en curso.
- **Implementación con modelos locales:** para sistemas que no tienen acceso a logprobs via API, usar modelos open source (Llama 3, Mistral, Qwen) desplegados con vLLM o Ollama que exponen logprobs en la respuesta; o implementar FLARE por prompting: instruir al modelo a rodear los segmentos de incertidumbre con marcadores especiales (`<incerto>` , `</incerto>`) en la generación tentativa y usar esos marcadores como señal de recuperación.

Nota del Arquitecto: *FLARE es el patrón avanzado con el ROI más claro en dominios donde las alucinaciones tienen consecuencias graves: diagnóstico médico, interpretación legal, análisis financiero regulatorio. En esos dominios, el costo de 2–4x en generación para detectar y corregir afirmaciones inciertas es trivial comparado con el costo de una afirmación incorrecta presentada como hecho al usuario. Para sistemas de soporte técnico general o QA sobre documentación interna, el RAG estándar con Corrective RAG lightweight provee calidad suficiente con menor overhead.*

FLARE opera sobre la señal interna de confianza del modelo para guiar la recuperación. El siguiente patrón —Agentic RAG— eleva esta idea a un ciclo completo de razonamiento autónomo multi-paso.

Módulo 6 – Capítulo 10 – Sección 05

Agentic RAG: integrar recuperación en ciclos de razonamiento multi-paso

Los patrones anteriores de este capítulo —Self-RAG, Adaptive RAG, Corrective RAG, FLARE— son extensiones del paradigma fundamentalmente reactivo del RAG: el sistema recibe una query, decide cómo recuperar información, recupera, y genera. Incluso cuando múltiples recuperaciones ocurren (Self-RAG con múltiples tokens `[Retrieve]`, FLARE con múltiples iteraciones de lookahead), el propósito de cada recuperación es responder la query original. Agentic RAG representa un salto cualitativo: el LLM ya no responde queries, **persigue objetivos** mediante un ciclo de razonamiento autónomo donde la recuperación de información es una de las herramientas disponibles para alcanzar el objetivo, y donde cada resultado de recuperación puede modificar el plan de acción del agente para los pasos siguientes.

La diferencia se hace concreta con un ejemplo. Una query como "Resume los cambios regulatorios en el sector financiero europeo entre 2022 y 2024 y su impacto en las entidades de menos de €1B de activos" no tiene una respuesta que pueda construirse con un único ciclo de recuperación. Un agente que persigue ese objetivo puede razonar: "Primero necesito identificar qué regulaciones financieras europeas se publicaron entre 2022 y 2024." Recupera información sobre ese tema, analiza los resultados, y continúa: "He encontrado tres regulaciones relevantes: DORA, MiCA y la revisión de EMIR. Ahora necesito entender el impacto específico de cada una sobre entidades pequeñas." Ejecuta tres búsquedas adicionales —una por regulación— analiza los resultados, identifica que DORA tiene una sección específica sobre entidades por tamaño, recupera esa sección, y finalmente sintetiza una respuesta coherente que integra cinco o seis recuperaciones encadenadas. Un sistema RAG estándar con una única búsqueda inicial no puede construir esta respuesta.

El **patrón ReAct** (Reasoning + Acting, Yao et al. 2022) es el ciclo de razonamiento más utilizado para Agentic RAG: el LLM alterna entre tres estados. En el estado Thought, el LLM razona sobre el estado actual del problema y lo que necesita saber para avanzar. En el estado Action, el LLM selecciona y ejecuta una herramienta —que puede ser búsqueda en el índice

vectorial, búsqueda web, consulta SQL, llamada a API externa— con los parámetros apropiados. En el estado *Observation*, el LLM lee el resultado de la herramienta y actualiza su comprensión del problema. El ciclo continúa hasta que el LLM determina que tiene suficiente información para generar la respuesta final, o hasta que se alcanza el número máximo de iteraciones. El patrón **plan-and-execute** es una variante más estructurada: un LLM planner genera primero un plan completo de acciones (qué recuperar, en qué orden, cómo combinar los resultados), un ejecutor realiza cada acción secuencialmente, y un sintetizador genera la respuesta final usando todos los resultados. El *plan-and-execute* es más predecible en el número de recuperaciones —el plan define el número desde el inicio— pero menos adaptable a resultados inesperados durante la ejecución.

LangGraph (LangChain) es actualmente el framework más adoptado para implementar *Agentic RAG* con complejidad de flujo controlada: permite definir el ciclo de razonamiento como un grafo dirigido con nodos (estados del agente) y edges (transiciones condicionales), con soporte nativo para checkpoints de estado que permiten interrumpir y reanudar el razonamiento. Los ciclos controlados en LangGraph tienen límites de iteración configurables que previenen la ejecución infinita. **LlamaIndex Workflows** y **AutoGen** (Microsoft) son alternativas para sistemas multi-agente donde múltiples LLMs especializados —un agente de búsqueda vectorial, un agente de búsqueda web, un agente de análisis SQL— colaboran para responder queries que requieren múltiples tipos de recuperación coordinados.

La **observabilidad** es especialmente crítica en *Agentic RAG*: los errores en ciclos de razonamiento multi-paso son difíciles de diagnosticar sin tracing completo de cada *Thought*, *Action* y *Observation*. Un agente que toma una decisión incorrecta en el tercer paso de un ciclo de siete pasos produce una respuesta incorrecta cuya causa raíz es invisible sin el registro de la traza completa. Langfuse y LangSmith registran automáticamente cada paso del ciclo *ReAct* como spans anidados con el texto de cada *Thought*, el nombre de la herramienta llamada, los argumentos de la llamada y el resultado de la *Observation*, produciendo trazas que permiten al equipo de ingeniería diagnosticar exactamente dónde el agente tomó la decisión incorrecta.

Patrones de implementación de *Agentic RAG*

- **ReAct para RAG:** ciclo *Thought* → *Action* (búsqueda RAG o herramienta) → *Observation* repetido hasta respuesta final o límite de iteraciones; implementado en LangChain con `create_react_agent`; límite de iteraciones máximo (3–5 pasos) para prevenir ciclos infinitos y controlar la latencia total de la respuesta.

- **Plan-and-execute RAG:** LLM planner genera un plan de búsqueda estructurado antes de ejecutar ninguna búsqueda; ejecutor realiza cada búsqueda del plan secuencialmente; sintetizador genera la respuesta final con todos los resultados; más predecible y auditable que ReAct, menos adaptable a resultados inesperados intermedios.
- **LangGraph para flujos complejos:** flujos de recuperación con branches condicionales ("si el chunk recuperado no es relevante, ejecutar búsqueda web de fallback"), ciclos controlados con límite de iteraciones y checkpoints de estado para interrumpir y reanudar; adecuado para pipelines de Agentic RAG con lógica de routing compleja que sería difícil de mantener como código imperativo.
- **Multi-agent RAG con especialización:** agentes especializados (agente de corpus vectorial, agente de búsqueda web, agente de consulta SQL, agente de APIs externas) coordinados por un orchestrator que asigna sub-queries al agente con la capacidad correcta; AutoGen y CrewAI implementan este patrón con soporte para comunicación entre agentes y coordinación de resultados.
- **Límite de iteraciones y fallback:** definir el número máximo de ciclos de recuperación por solicitud (3–5 iteraciones es el rango práctico para sistemas de producción); si se alcanza el límite sin completar el objetivo, generar una respuesta parcial con la información disponible y señalarlo explícitamente al usuario en lugar de continuar indefinidamente.
- **Observabilidad de ciclos de razonamiento:** instrumentar cada Thought, Action y Observation como spans de OpenTelemetry o como eventos en Langfuse/LangSmith; el tracing completo del ciclo es imprescindible para diagnosticar errores de razonamiento y para analizar qué tipos de decisiones del agente producen respuestas incorrectas.

Nota del Arquitecto: Agentic RAG introduce tres desafíos operacionales que no existen en RAG estándar: latencia variable (el número de ciclos determina el tiempo de respuesta, que puede variar de 2 a 30 segundos para la misma query), debugging de errores de razonamiento (identificar en qué paso del ciclo el agente tomó la decisión incorrecta requiere trazas completas), y costo impredecible por solicitud (cada ciclo adicional tiene un costo LLM). Estos desafíos son manejables con LangGraph + Langfuse + límites de iteración bien calibrados, pero justifican reservar Agentic RAG para los casos de uso que genuinamente requieren razonamiento multi-paso: investigación, análisis comparativo profundo, o workflows que necesitan coordinar múltiples tipos de fuentes de información.

Con Agentic RAG, el paradigma RAG alcanza su forma más sofisticada: el sistema no solo recupera información, sino que razona sobre qué información necesita, en qué orden, y cómo integrarla. El capítulo de cierre sintetiza el conjunto de patrones avanzados y establece la conexión con el siguiente módulo del libro.

Módulo 6 – Capítulo 10 – Sección 06

Cierre: los patrones avanzados de RAG son respuestas a limitaciones concretas del sistema

Self-RAG, Adaptive RAG, Corrective RAG, FLARE y Agentic RAG no son innovaciones académicas desconectadas de la práctica ni patrones que deben implementarse juntos para tener un "RAG moderno". Son respuestas ingenieriles a problemas concretos y frecuentes que los equipos de producción encuentran en sus sistemas RAG: el RAG estándar recupera siempre aunque a veces el modelo ya tiene la respuesta, desperdiciando latencia y costo en recuperaciones innecesarias (Self-RAG lo resuelve). El mismo pipeline de recuperación es ineficiente para la diversidad de tipos de queries que recibe un sistema de propósito amplio (Adaptive RAG lo resuelve). El retriever devuelve chunks de baja relevancia que contaminan el contexto del generador produciendo alucinaciones (Corrective RAG lo resuelve). El modelo necesita información adicional específica en mitad de la generación que la recuperación inicial no anticipó (FLARE lo resuelve). Algunas queries requieren múltiples ciclos secuenciales de recuperación y razonamiento para construir una respuesta completa (Agentic RAG lo resuelve).

El **criterio de adopción** de cada patrón debe ser empírico y específico al sistema: identificar, mediante análisis de logs y métricas de evaluación, el tipo de error más frecuente en el sistema, y adoptar el patrón que específicamente resuelve ese tipo de error. Adoptar Agentic RAG sin haber identificado que las queries multi-step son frecuentes es sobreingeniería que añade complejidad operacional sin mejora de calidad medible. Ignorar Corrective RAG cuando el análisis muestra que el 20% de las respuestas incluyen alucinaciones causadas por recuperación fallida es negligencia de ingeniería. La decisión correcta para cada sistema está en los datos de evaluación, no en el paper más reciente sobre patrones avanzados de RAG.

La **adopción incremental** sigue siendo el principio operacional más seguro también en el contexto de los patrones avanzados: empezar con el RAG estándar bien instrumentado, establecer métricas de evaluación sólidas, identificar los fallos más frecuentes mediante análisis sistemático, y adoptar exactamente el patrón que resuelve el fallo dominante. Esta

estrategia es superior a la estrategia alternativa de implementar desde el inicio el sistema más sofisticado posible, porque cada patrón adicional añade complejidad de debugging, overhead de latencia, costo operacional y superficie de fallos que debe gestionarse en producción. La complejidad correcta del sistema es la mínima que alcanza los SLOs de calidad definidos por los usuarios.

De RAG a agentes: el puente hacia el Módulo 7

Agentic RAG es el patrón donde la frontera entre un sistema RAG sofisticado y un sistema de agentes de IA se vuelve borrosa —y esa borrosidad es intencional e instructiva. Un agente de IA es, en esencia, un sistema que usa RAG y herramientas de forma autónoma para completar tareas que requieren múltiples pasos de razonamiento y acción. La recuperación de información del corpus, que este módulo ha presentado como el núcleo del paradigma RAG, es en el contexto de los agentes una herramienta más —la misma herramienta de búsqueda vectorial que se construyó en los Capítulos 2, 3 y 4 de este módulo— disponible en el conjunto de capacidades que el agente puede usar para perseguir sus objetivos.

Lo que el Módulo 7 desarrollará —arquitectura de agentes, patrones de orquestación multi-agente, gestión de memoria persistente, herramientas de ejecución de código, planificación de largo plazo— es la generalización de los principios que este módulo estableció para el caso específico de la recuperación de información. Un agente que escribe código para analizar un dataset, busca documentación sobre la librería que está usando, corrige los errores que el intérprete devuelve y genera un reporte con los resultados está ejecutando exactamente el ciclo Agentic RAG de este capítulo, con el conjunto de herramientas ampliado para incluir la ejecución de código. Los sistemas RAG que construiste en este módulo —el pipeline de ingesta, el índice vectorial, el retriever híbrido con reranker, el evaluador RAGAS, la arquitectura de producción desacoplada— son los componentes del sistema de memoria y conocimiento que los agentes del Módulo 7 usarán como sustrato.

El AI Engineer que domina la ingeniería de sistemas RAG tiene la base más sólida posible para construir sistemas de agentes: sabe cómo indexar y recuperar conocimiento con calidad medida, cómo evaluar la calidad de la información que llega al modelo, cómo operar el sistema de conocimiento en producción con la confiabilidad que los agentes de largo plazo requieren. Con esa base, la transición a la ingeniería de agentes de IA es una extensión natural del trabajo que este módulo enseñó.

"The art of making systems simple again is harder than making complex ones. It takes more skill, more insight, and more courage." — Leslie Lamport, Premio Turing 2013, sobre el

diseño de sistemas distribuidos correctos y simples; igualmente válido para sistemas de AI Engineering en producción.